CASE 1 • TOKENIZATION OF REAL-WORLD ASSETS • SOLANA DEVNET MVP

Техническое задание

RWA Receivables

MVP-проект по токенизации подтверждённых инвойсов / дебиторской задолженности на Solana

Параметр	Содержание
Основание	Кейс 1 National Solana Hackathon by Decentrathon: Tokenization of Real-World Assets (RWA)
Назначение документа	Implementation-ready ТЗ: продукт, архитектура, backlog, команда, acceptance criteria
Формат	Hackathon MVP, Solana devnet, web-first demo
Основной актив	Подтверждённый инвойс / дебиторская задолженность, расчёты в USDC
Версия	v1.0
Дата	29 марта 2026

Что должно получиться на демо

- Issuer создаёт актив на основе инвойса; verifier подтверждает документы и открывает funding.
- Два allowlisted инвестора покупают доли актива за USDC, затем выполняется один secondary transfer.
- Settlement operator фиксирует repayment; держатели токена делают claim payout, а актив корректно закрывает lifecycle.
- Веб-интерфейс показывает статус актива, историю действий и ключевые on-chain события.

Этот документ намеренно сужает scope. Мы не строим универсальную RWA-платформу и не пытаемся юридически заменить факторинговую инфраструктуру во всех юрисдикциях. Мы делаем узкий, честный и демонстрируемый MVP под критерии кейса 1: deployed program, выпуск токенов, управление владением/транзакциями, базовый интерфейс и связка токена с реальным активом.

Содержание

№	Раздел	Что внутри
1	Основание и цель документа	Зачем нужен этот ТЗ и какую задачу он решает
2	Продуктовая рамка	Проблема, гипотеза, ценность и ключевое допущение MVP
3	Scope MVP	Что входит, что не входит, какие ограничения намеренно вводятся
4	Роли и end-to-end user flow	Кто участвует в системе и как проходит жизненный цикл актива
5	Архитектура решения	On-chain и off-chain контур, компоненты и технические принципы
6	On-chain design	Аккаунты, статусы, инструкции, токеномика и инварианты
7	Backend specification	API, сервисы, БД, интеграции, требования к серверной части
8	Frontend specification	Экраны, состояния, UX-принципы и пользовательские сценарии
9	Разбивка работы: backend / frontend	Подробный backlog, зависимости и definition of done
10	Спринт-план и контрольные точки	Реалистичный план работы команды на хакатон
11	Acceptance criteria и демо-сценарий	Что обязательно показать судьям и как проверять готовность
12	Риски, репозиторий и приложения	Риски, структура GitHub, пример demo asset и техническая база

Жёсткая рамка MVP

- Один тип актива: invoice / receivable.
- Один расчётный актив: USDC на Solana.
- Один proof-of-asset flow: document hash + verifier attestation + payment evidence hash.
- Один secondary circulation path: transfer только между allowlisted wallet-ами.

1. Основание и цель документа

Документ подготовлен на основе кейса 1 "Tokenization of Real-World Assets (RWA)" из технического задания National Solana Hackathon by Decentrathon. В исходном документе от участников требуется использовать Solana blockchain, задеплоить program, создавать/выпускать токены, управлять владением или транзакциями, иметь базовый интерфейс и показать связь между реальным активом и цифровым токеном через oracle, off-chain данные или иной механизм подтверждения.

1.1. Назначение ТЗ

Цель этого ТЗ - убрать двусмысленность и превратить идею проекта в план реализации, по которому команда может сразу распределять ответственность, писать код и готовить демонстрацию. Документ задаёт продуктовую рамку, архитектуру, модель данных, API, экраны интерфейса, технические ограничения, а также подробную разбивку работ на backend и frontend.

1.2. Почему выбран именно invoice RWA

Из всех типовых RWA-идей именно токенизация подтверждённых инвойсов даёт лучший баланс между реальной экономической болью, ясным user flow и реалистичностью хакатонного MVP. Недвижимость слишком тяжёлая по legal surface, сырьё и энергия требуют сложного proof-of-asset контура, а "универсальный конструктор RWA" почти всегда превращается в расплывчатую демо-обёртку. Инвойс же позволяет показать полный цикл: выпуск актива -> финансирование -> владение -> ограниченный transfer -> погашение -> payout.

Ключевой принцип

- Мы не обещаем заменить юридический факторинг во всех странах.
- Мы честно показываем on-chain модель владения и распределения payout поверх верифицированного off-chain актива.
- Сильная сторона проекта - не абстрактная уникальность, а убедительный end-to-end flow на Solana.

2. Продуктовая рамка

2.1. Проблема

Малый и средний бизнес часто выставляет инвойсы с отсрочкой оплаты на 30-90 дней. В этот промежуток компания уже выполнила поставку или услугу, но ещё не получила деньги, и из-за этого испытывает кассовый разрыв. Классический факторинг решает проблему ликвидности, но обычно остаётся непрозрачным, зависимым от посредников и плохо приспособленным к дробному участию инвесторов в отдельных сделках.

2.2. Решение

RWA Receivables превращает подтверждённый инвойс в токенизированный актив на Solana. После проверки документов система выпускает отдельный Token-2022 mint на каждый инвойс. Инвесторы из allowlist покупают доли актива за USDC, а после off-chain погашения инвойса payout распределяется on-chain пропорционально фактическому владению токенами.

2.3. Ключевое допущение

В рамках MVP токен не декларируется как универсально юридическое право собственности на дебиторскую задолженность в любой юрисдикции. Токен означает долю в on-chain payout-модели, привязанной к конкретному подтверждённому инвойсу и обслуживаемой через verifier/attestor модель.

2.4. Продуктовая гипотеза

- Issuer получает ликвидность раньше срока оплаты и видит прозрачный статус сделки.
- Investor получает short-duration real-world yield инструмент с понятной математикой payout.
- Solana даёт масштабируемую on-chain основу для учёта владения, restricted transfers и автоматизированного settlement.
- Для хакатона важнее продемонстрировать узкий, работающий и честный сценарий, чем пытаться покрыть весь рынок trade finance.

2.5. Ценность по ролям

Сторона	Проблема	Ценность продукта
Issuer	Кассовый разрыв до даты оплаты инвойса	Получает ликвидность раньше, без непрозрачной ручной схемы учёта
Investor	Нет простого short-term RWA инструмента с прозрачным lifecycle	Покупает дробные доли подтверждённого актива и получает payout on-chain
Verifier / Platform	Сложно показать реальную связь между активом и токеном	Использует document hash, attestation и event log как проверяемую цепочку состояний

3. Scope MVP

3.1. Что входит в MVP

- Только USD-denominated invoices; расчёты исключительно в USDC on Solana.
- Один актив = один инвойс = один отдельный mint с фиксированным supply.
- Ручная off-chain проверка документов verifier-ом.
- Allowlist инвесторов и restricted transfers через Transfer Hook.
- Funding round, secondary transfer между allowlisted участниками, repayment record, claim payout и refund для неуспешного funding.
- Веб-интерфейс для Issuer, Investor, Verifier и Admin/Settlement operator.

3.2. Что сознательно не входит

- Open marketplace без allowlist и без ограничений оборота.
- Полноценный order book / AMM / secondary market matching engine.
- Частичные repayment-транши, сложное waterfall-распределение и tranche-структура.

- Автоматические банковские интеграции дебитора и trustless settlement.
- Поддержка нескольких валют, FX и multi-chain инфраструктуры.
- Юридическая унификация уступки требования для разных стран.

Почему такая жёсткая фокусировка правильна

- Критерии кейса оценивают Product, Technical Implementation и Use of Solana сильнее, чем абстрактную широту охвата.
- Каждая дополнительная ветка логики увеличивает риск провалить demo в самый важный момент.
- Сильный MVP здесь - это один актив, один сценарий и одна безупречная демонстрация жизненного цикла.

4. Роли и end-to-end user flow

4.1. Роли в системе

Роль	Описание	Ключевые действия	Доступ
Issuer	Компания, которой нужна ликвидность под инвойс	Создать актив, загрузить документы, отслеживать funding и payout status	Wallet + личный кабинет issuer
Investor	Покупает доли актива за USDC и получает payout	Купить asset tokens, переводить allowlisted контрагенту, claim payout, получить refund	Wallet + portfolio
Verifier	Проверяет документы и управляет allowlist	Подтверждать активы, открывать funding, вносить KYC/allowlist	Verifier dashboard
Admin / Settlement Operator	Управляет platform config и фиксирует repayment	Настройка платформы, pause, payment recording, финализация актива	Admin dashboard
Debtor (off-chain)	Внешняя сторона, оплачивающая инвойс	Оплачивает инвойс вне приложения	Не является пользователем UI в MVP

4.2. End-to-end сценарий

Шаг	Актор	Действие	Результат
1	Issuer	Создаёт карточку актива и загружает пакет документов	Asset получает статус Created; on-chain фиксируется asset record с hash/URI

Шаг	Актор	Действие	Результат
2	Verifier	Проверяет документы и подтверждает актив	Актив становится Verified; mint и vault-ы готовы к инициализации
3	Admin/Verifier	Инициализирует Token-2022 mint и открывает funding	Статус FundingOpen; полный supply токенов хранится в program vault
4	Investor A / B	Покупают доли актива за USDC	USDC поступает в funding vault; токены передаются инвесторам
5	Investor	Делает secondary transfer другому allowlisted holder-у	Transfer проходит только при соответствии правилам Transfer Hook
6	Admin	Закрывает funding и переводит средства issuer-у	Актив переходит в Funded; issuer получает финансирование
7	Settlement Operator	После реальной оплаты инвойса записывает repayment + evidence hash	Статус Paid; transfers блокируются, payout vault наполнен USDC
8	Investor	Выполняет claim payout	Программа сжигает asset tokens и переводит инвестору USDC пропорционально балансу
9	Admin	Финализирует актив после нулевого outstanding supply	Статус Closed

5. Архитектура решения

5.1. Технические принципы

- On-chain хранит правила, состояния и владение; off-chain обслуживает документы, review и attestation.
- Каждый инвойс - отдельный актив и отдельный mint, чтобы не размазывать риски и не смешивать сделки.
- Transfer restrictions контролируются программно, а не только интерфейсом.
- Все денежные значения считаются в integer base units; никакой бизнес-логики на float.
- Документ не должен требовать доверия к устным обещаниям: каждое ключевое изменение состояния оставляет on-chain event и машиночитаемый след.

5.2. Компонентная схема

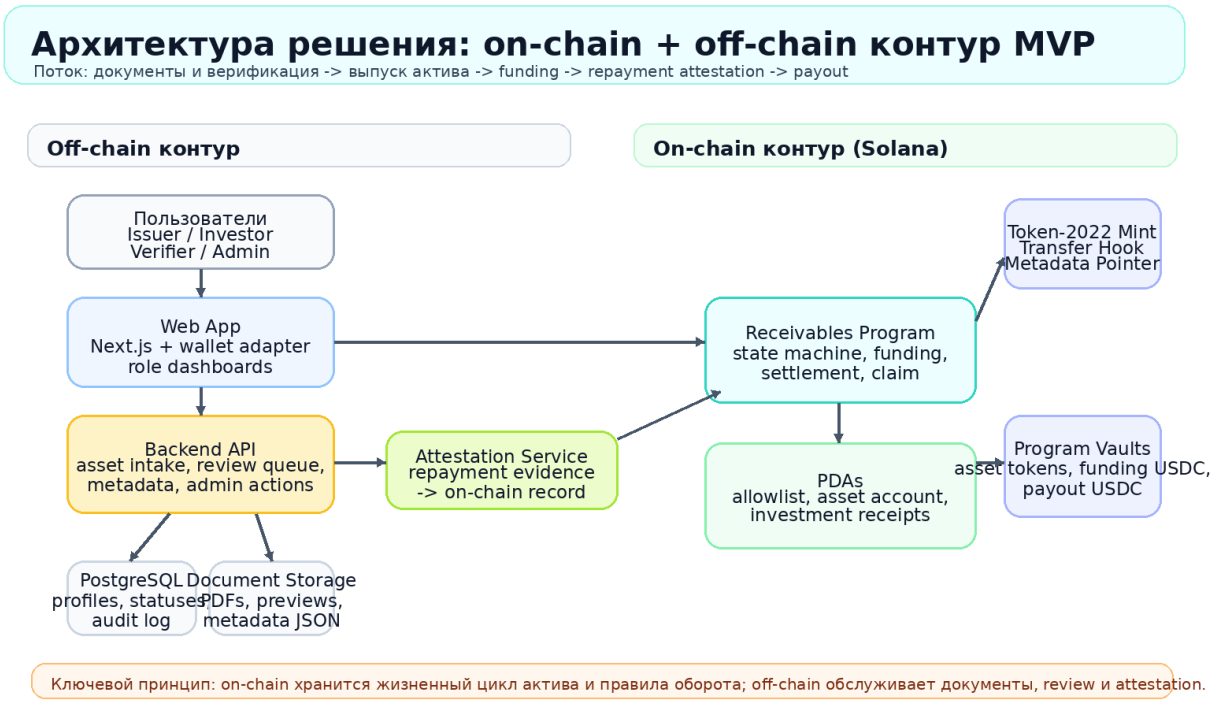

Схема 1. Компоненты решения и движение данных между off-chain и on-chain контуром

5.3. Компоненты системы

Компонент	Назначение	Предлагаемая технология
Receivables Program	State machine актива, funding, refund, repayment, payout claim	Rust + Anchor
Transfer Hook Program	Проверка restricted transfer между allowlisted wallet-ами	Rust + Anchor / native Solana program
Token-2022 mint	Mint актива с metadata pointer и transfer hook	Token Extensions
Web App	Интерфейс всех ролей, wallet connection, транзакционные сценарии	Next.js + React + Solana wallet adapter
Backend API	Asset intake, review queue, metadata, admin flows, seed/demo automation	Node.js / NestJS или Next API
Database	Профили, роли, статусы, журнал действий, кэш данных для UI	PostgreSQL
Document storage	PDF-файлы, preview, metadata JSON	IPFS/Pinata или S3-compatible storage
Attestation service	Repayment evidence -> on-chain record_payment	Node worker / signed service wallet

6. On-chain design

6.1. Базовая модель аккаунтов

Аккаунт / PDA	Назначение	Ключевые поля
PlatformConfigPDA	Глобальный конфиг платформы	admin, verifier_authority, attestor_authority, usdc_mint, platform_fee_bps, paused
AssetPDA	Главная сущность актива	issuer, invoice_hash, metadata_uri, debtor_ref_hash, face_value, discount_bps, funding_target, due_date, status, mint
WhitelistEntryPDA	Allowlist запись по кошельку	wallet, role_mask, active, kyc_ref_hash, updated_at
InvestmentReceiptPDA	Учёт вклада конкретного инвестора в funding round	asset, investor, contributed_usdc, received_asset_tokens, refunded
AssetTokenVault ATA	Vault asset token-ов под контролем программы	Minted supply до момента покупки
FundingVault USDC ATA	Приём средств инвесторов	USDC до settlement в issuer / refund
PayoutVault USDC ATA	Хранение payout средств после repayment	USDC для claim payout

6.2. Модель статусов актива

Статус	Смысл	Разрешённые переходы
Created	Актив создан issuer-ом, но ещё не подтверждён	-> Verified
Verified	Документы подтверждены, mint может быть инициализирован	-> FundingOpen
FundingOpen	Открыт первичный сбор средств	-> Funded или -> Cancelled
Funded	Funding round закрыт успешно, issuer получил средства	-> Paid
Paid	Repayment подтверждён, payout vault наполнен	-> Closed
Cancelled	Funding не удался; доступен refund	terminal
Closed	Lifecycle завершён, outstanding supply = 0	terminal

6.3. Инструкции программы

Instruction	Кто вызывает	Назначение	Ключевые проверки
initialize_platform	Admin	Создание platform config	Однократная инициализация
upsert_whitelist_entry	Admin / Verifier	Добавление и обновление allowlist	Право роли, валидность wallet
create_asset	Issuer	Создание AssetPDA	Корректность дат и параметров
verify_asset	Verifier / Admin	Подтверждение актива	status = Created; due_date > funding_deadline
initialize_asset_mint	Admin / Verifier	Создание Token-2022 mint и vault-ов	Расширения mint заданы заранее
open_funding	Admin / Verifier	Открыть funding round	status = Verified
buy_primary	Investor	Купить долю актива за USDC	allowlist, deadline, supply, valid status
close_funding	Admin / Verifier	Закрыть funding	Target reached или deadline passed
settle_funding_to_issuer	Program / Admin	Перевести собранные средства issuer-у	status = Funded
refund	Investor	Вернуть деньги при Cancelled	not refunded, status = Cancelled
record_payment	Attestor / Admin	Зафиксировать repayment и evidence hash	status = Funded; payout vault funded
claim_payout	Investor	Получить выплату пропорционально балансу	status = Paid; balance > 0
finalize_asset	Admin	Закрыть lifecycle	Outstanding supply = 0

6.4. Модель токена и payout math

Для простоты демонстрации принимается правило 1 asset token = 1 USDC nominal claim. Mint имеет 6 decimals. Если nominal value инвойса составляет 10,000 USDC, total supply равен 10,000.000000 INV. При discount 95% инвестор покупает 1,000 INV за 950 USDC, а после погашения инвойса получает 1,000 USDC. Такая модель интуитивна и резко упрощает объяснение payout судьям.

Инварианты безопасности

- После статуса Verified ключевые параметры актива не редактируются.
- Покупка и transfer доступны только allowlisted участникам.
- После статуса Paid любые transfer-ы блокируются, а claim выполняется через burn asset token-ов.
- Refund и claim должны быть идемпотентными: повторное выполнение сверх

допустимого недоступно.
- Все vault-аккаунты находятся под program authority через PDA; пользователь не может произвольно выводить средства.

7. Backend specification

7.1. Границы backend-контура

Под backend-частью в рамках этого ТЗ понимаются два связанных, но разных потока: (а) Backend/Blockchain - on-chain программы, тесты, deploy scripts, индексирование событий; (б) Backend/API - серверная логика вокруг документов, metadata, review queue, database, admin actions и automation. Такое разделение важно: иначе смарт-контрактная работа начинает "съедать" сроки web/API части, а к демо команда остаётся без полноценного управляемого интерфейса.

7.2. Backend API и сервисные модули

Модуль	Ответственность	Ключевые операции
Assets service	Создание драфтов, хранение статусов, orchestration create/verify/open funding	create asset draft, get asset, list marketplace
Documents service	Приём файлов, checksum/hash, metadata bundle	upload docs, preview, generate evidence hash
Verification service	Review queue и действия verifier-а	approve/reject, reason log, allowlist update
Whitelist service	Связка роли, KYC ref и wallet	add/remove wallet, sync on-chain entry
Settlement service	Payment evidence и on-chain repayment record	record payment, verify payout funding
Indexer / events service	Чтение событий Solana и кэширование для UI	asset history, portfolio refresh, activity feed
Demo seed service	Развёртывание тестовых данных для демо	seed wallets, seed asset, scripted flow
Audit log service	Наблюдаемость и диагностика	action logs, errors, admin traces

7.3. Предлагаемый API-контракт

Метод	Endpoint	Описание	Роль
POST	/api/assets	Создать asset draft	Issuer
POST	/api/assets/:id/documents	Загрузить пакет документов	Issuer
GET	/api/assets/:id	Получить карточку актива	Any authenticated role
GET	/api/marketplace	Список funding/funded	Investor

Метод	Endpoint	Описание	Роль
		активов	
GET	/api/portfolio/:wallet	Портфель пользователя	Investor
GET	/api/review-queue	Очередь на проверку	Verifier
POST	/api/assets/:id/verify	Подтвердить актив	Verifier/Admin
POST	/api/whitelist	Добавить или обновить allowlist entry	Verifier/Admin
POST	/api/assets/:id/open-funding	Открыть funding round	Verifier/Admin
POST	/api/assets/:id/close-funding	Закрыть funding round	Verifier/Admin
POST	/api/assets/:id/record-payment	Зафиксировать repayment и evidence hash	Admin/Attestor
GET	/api/activity/:assetId	История событий по активу	All relevant roles

7.4. Модель данных off-chain

Таблица	Ключевые поля	Назначение
users	wallet, role, display_name, active	Кэш ролей и профилей для UI
assets	asset_id, issuer_wallet, status, face_value, due_date, mint, metadata_uri	Основной off-chain кэш состояния
documents	asset_id, filename, file_uri, content_hash, kind	Пакет файлов по активу
review_actions	asset_id, verifier_wallet, decision, comment, created_at	История review и причины отклонений
whitelist_entries	wallet, role_mask, kyc_ref_hash, active	Оффчейн-представление allowlist
payments	asset_id, evidence_hash, amount, operator_wallet, created_at	История settlement действий
activity_log	entity_type, entity_id, action, payload, tx_sig	Диагностика и audit trail

7.5. Нефункциональные требования к backend

- Все бизнес-расчёты и все on-chain суммы - только в integer base units.
- Idempotency для критических серверных действий: повторный запрос не должен ломать состояние.
- Наличие health-check, seed script, demo reset script и понятного `.env.example`.

- Логи должны позволять быстро восстановить цепочку: API action -> wallet -> tx signature -> result.
- Devnet deployment, unit tests и минимальный integration script обязательны до финального демо.

7.6. Definition of Done для backend

- Program-ы задеплоены, program IDs зафиксированы в README и env-конфигах.
- Все критические инструкции покрыты happy-path тестами и минимум 3 негативными кейсами.
- API возвращает стабильные данные для marketplace, asset detail, review queue и portfolio.
- Есть демо-скрипт, позволяющий за 2-3 минуты поднять и показать опорный сценарий.

8. Frontend specification

8.1. UX-принципы

- Интерфейс должен объяснять продукт без голосового сопровождения. Судья должен понимать статус актива, кто что сделал и что произойдёт дальше.
- Каждая транзакция должна иметь понятное состояние: ready -> pending -> confirmed -> failed.
- Ролевой доступ должен отражаться в UI: issuer не видит verifier operations, investor не видит admin-only controls.
- Главный KPI фронтенда - не красота сама по себе, а прозрачность user flow и отсутствие путаницы в critical actions.

8.2. Обязательные экраны

Экран	Основные действия	Ключевые данные / состояния
Marketplace	Просмотр активов, фильтрация, переход в карточку	Funding progress, nominal value, due date, expected yield, status
Asset Detail	Просмотр параметров и покупка доли	Metadata, document hash, history, Invest CTA, tx states
Submit Asset	Создание актива и загрузка документов	Form validation, upload progress, draft status
Portfolio	Просмотр владения, transfer, claim payout, refund	Holdings, avg price, balance, status badges
Verifier Dashboard	Review queue, verify/reject, whitelist actions	Document list, decision log, allowlist status
Admin / Settlement Dashboard	Open/close funding, record payment, pause, finalize	Funding metrics, payout readiness, admin actions

8.3. Ключевые frontend flows

1. Wallet connection и определение роли: приложение читает wallet, роль и allowlist status; при отсутствии доступа показывает информативный gated state, а не пустой экран.
2. Invest flow: investor открывает asset detail, вводит сумму, подтверждает транзакцию USDC + transfer asset token-ов и видит подтверждённый результат вместе с новым балансом.
3. Transfer flow: holder указывает wallet получателя; UI предупреждает, что перевод возможен только allowlisted участнику, а итоговый успех зависит от on-chain hook validation.
4. Claim flow: после статуса Paid UI показывает точную сумму claim и после подтверждения обновляет both token balance и USDC balance.
5. Verifier/Admin flow: отдельные панели для review и settlement должны убирать лишние поля и подсвечивать критические условия, чтобы избежать ошибок на демо.

8.4. Definition of Done для frontend

- Все обязательные экраны реализованы и подключены к живым данным, а не к статическим mock-объектам.
- Все транзакционные кнопки имеют pending/success/error state и понятные сообщения.
- Есть единый стиль статусов: Created, Verified, FundingOpen, Funded, Paid, Cancelled, Closed.
- Демо можно провести без ручных правок в коде прямо перед выступлением.

9. Разбивка работы: backend / frontend

Ниже - реалистичная декомпозиция для команды из 4-5 человек. При меньшем составе Backend/API и Frontend частично совмещаются, но split по deliverables остаётся тем же. Самая частая ошибка на хакатонах - считать, что "backend = просто контракт". Нет: смарт-контракт без API, review-панели, storage и удобной orchestration-логики не превращается в продукт.

9.1. Предлагаемая команда

Роль	Фокус	Основной результат
Tech lead / PM	Архитектура, интеграция, demo ownership	Статус проекта, критические решения, репетиция демо
Backend / Blockchain	Anchor programs, Token-2022, scripts, tests	Рабочая on-chain логика и deploy
Backend / API	DB, storage, admin flows, indexer, seed scripts	Живые данные и управляемый серверный слой
Frontend	Marketplace, dashboards, wallet flows, polish	Понятный и стабильный web demo
QA / Fullstack (optional)	Склейка потоков, regression check	Быстрый проход по багам и ux polish

9.2. Backend / Blockchain backlog

Блок	Задачи	Артефакт	Оценка
BC-01	Поднять Anchor workspace, конфиги devnet, локальные ключи ролей, program IDs	Bootstrap repo + env	0.5 дн
BC-02	Спроектировать account model и enum статусов	IDL + account structs	0.5 дн
BC-03	Реализовать initialize_platform и upsert_whitelist_entry	Whitelist flow	0.5 дн
BC-04	Реализовать create_asset / verify_asset с проверками дат и immutable params после verify	Asset lifecycle part 1	0.75 дн
BC-05	Инициализация Token-2022 mint, metadata pointer, mint authority, vault accounts	Asset mint flow	0.75 дн
BC-06	Открытие funding и buy_primary с учётом USDC transfer и investment receipt	Primary funding flow	1.0 дн
BC-07	close_funding + settle_funding_to_issuer + refund unhappy path	Funding close/refund	0.75 дн
BC-08	Transfer Hook validation для secondary transfer между allowlisted wallet-ами	Restricted secondary transfer	1.0 дн
BC-09	record_payment + claim_payout + finalize_asset	Settlement flow	0.75 дн
BC-10	Тесты happy/negative path, scripted demo transactions	Test suite + scripts	1.0 дн

9.3. Backend / API backlog

Блок	Задачи	Артефакт	Оценка
API-01	Создать базовый сервис, env config, health-check, role config, seed wallets	API skeleton	0.5 дн

Блок	Задачи	Артефакт	Оценка
API-02	Реализовать upload документов, hashing, metadata JSON и хранение URI	Documents pipeline	0.75 дн
API-03	Создать таблицы users/assets/documents/review_actions/payments/activity_log	DB schema + migrations	0.5 дн
API-04	Assets service: create draft, get asset, list marketplace	Core asset endpoints	0.5 дн
API-05	Verification queue и whitelist management endpoints	Verifier API	0.5 дн
API-06	Settlement service + evidence hash + admin orchestration	Payment recording flow	0.5 дн
API-07	Indexer / event reader для истории актива и портфеля	Read-model for UI	0.75 дн
API-08	Demo seed and reset scripts, sample asset bootstrap	Demo automation	0.5 дн
API-09	Audit logs, error normalization, fallback data для demo diagnostics	Observability	0.5 дн

9.4. Frontend backlog

Блок	Задачи	Артефакт	Оценка
FE-01	Создать app shell, theme, wallet connection, route guards и role-aware layout	Base app shell	0.5 дн
FE-02	Marketplace page + asset cards + empty/loading/error states	Landing/marketplace	0.5 дн
FE-03	Asset detail page с metadata, history, invest widget и tx feedback	Asset detail flow	0.75 дн
FE-04	Issuer submit asset form + document upload UX	Submit asset flow	0.75 дн
FE-05	Portfolio page с	Investor dashboard	0.75 дн

Блок	Задачи	Артефакт	Оценка
	holdings, transfer modal, claim/refund actions		
FE-06	Verifier dashboard: review queue, docs preview, verify/reject, whitelist	Verifier UI	0.75 дн
FE-07	Admin/Settlement dashboard: open/close funding, record payment, pause/finalize	Admin UI	0.75 дн
FE-08	Unified status badges, activity timeline, transaction toasts, error mapping	UX polish layer	0.5 дн
FE-09	Final QA pass: responsive fixes, demo mode, seeded walkthrough	Presentation-ready UI	0.5 дн

9.5. Что можно делать параллельно, а что нельзя

Категория	Можно параллелить	Критическая зависимость
Параллельно	Front-end shell, marketplace UI, DB schema, upload flow, account model draft	Не требуют готового full happy path
Нельзя отложить	Account model, asset status machine, Token-2022 mint decisions	Их поздняя смена ломает и UI, и API, и тесты
Интеграционный стоп-фактор	buy_primary, transfer hook, payout claim	Без них нет доказательства end-to-end user flow
Финишный риск	Ручные demo hacks вместо seed scripts	На сцене это почти всегда ведёт к поломке

Definition of Ready для интеграции потоков

- Program ID, wallet roles и env-конфиги согласованы между всеми участниками.
- Asset status enum и поля AssetPDA больше не меняются без согласования с frontend/API.
- Есть хотя бы один seeded asset и два seeded investor wallet-а для ежедневной интеграции.

10. Спринт-план и контрольные точки

Ниже - агрессивный, но реалистичный план на 5 полноценных рабочих дней. Он предполагает ежедневную интеграцию, а не "склейку всего в последний вечер". Это

принципиально: для хакатона провал обычно происходит не из-за отсутствия идеи, а из-за слишком поздней синхронизации потоков.

День	Backend / Blockchain	Backend / API	Frontend	Выход дня
День 1	Account model, whitelist, create_asset	API skeleton, DB schema, upload flow skeleton	App shell, wallet connection, marketplace scaffold	Команда согласовала модель сущностей и env
День 2	verify_asset, mint init, vaults	Documents hashing, metadata, asset endpoints	Asset detail, submit asset form	Можно создать и верифицировать актив end-to-end
День 3	open_funding, buy_primary, close_funding	Review queue, whitelist API, seeded data	Invest flow, verifier dashboard	Funding happy path проходит без ручных костылей
День 4	Transfer hook, record_payment, claim_payout	Indexer/events, payment orchestration	Portfolio, transfer modal, admin dashboard	Весь жизненный цикл закрыт на devnet
День 5	Негативные кейсы, tests, deploy sanity	Demo reset script, logs, fallback diagnostics	UX polish, bug fix, presentation mode	Готовый demo script и репетиция выступления

10.1. Контрольные точки

- CP-1: к концу дня 2 команда обязана создать и подтвердить asset без ручной правки on-chain данных.
- CP-2: к концу дня 3 инвестор обязан купить токены через UI на devnet.
- CP-3: к середине дня 4 transfer hook должен реально отклонять heallowlisted получателя.
- CP-4: к концу дня 4 payout claim обязан менять both token и USDC balances корректно.
- CP-5: день 5 - только polish, diagnostics и репетиция. Не переписывание архитектуры.

11. Acceptance criteria и демо-сценарий

11.1. Обязательные acceptance criteria

Критерий	Проверка
AC-01	Issuer может создать новый asset и загрузить документы; UI отображает Created status.
AC-02	Verifier подтверждает asset; статус меняется на Verified, mint/vault flow инициализируется корректно.
AC-03	Funding открывается и минимум два инвестора покупают токены за USDC на devnet.
AC-04	Один secondary transfer между allowlisted инвесторами проходит успешно.

Критерий	Проверка
AC-05	Попытка перевода токена на неallowlisted wallet завершается ошибкой on-chain.
AC-06	Settlement operator фиксирует repayment и evidence hash; актив переходит в Paid.
AC-07	Каждый holder выполняет claim payout; balance changes соответствуют формуле payout.
AC-08	После полного claim outstanding supply становится нулевым, актив можно закрыть в Closed.
AC-09	README содержит архитектуру, setup, program IDs, env requirements и demo script.

11.2. Рекомендуемый demo asset

Параметр	Значение
Invoice nominal	10,000 USDC
Discount	95%
Funding target	9,500 USDC
Investor A buy	6,000 INV
Investor B buy	4,000 INV
Secondary transfer	Investor A -> Investor B: 1,000 INV
Repayment amount	10,000 USDC
Expected payouts	Investor A = 5,000 USDC; Investor B = 5,000 USDC

11.3. Demo narration order

- Показать marketplace и проблему, которую решает продукт, за 20-30 секунд.
- Открыть готовый asset, кратко объяснить, что хранится on-chain, а что off-chain.
- Показать верификацию / allowlist / funding.
- Провести purchase двумя инвесторами и один restricted transfer.
- Записать repayment, затем выполнить claim payout.
- Завершить объяснением того, почему именно Solana здесь используется по сути, а не для галочки.

12. Риски, структура репозитория и приложения

12.1. Основные риски и меры снижения

Риск	Как проявляется	Митигирующее действие
Transfer Hook затянется	Secondary transfer не работает к интеграции	Сначала сделать allowlist проверку и primary flow; hook интегрировать сразу после happy funding path
Смена account model в конце	Ломаются API, UI и тесты одновременно	Заморозить AssetPDA schema не позднее конца дня 1
Нет seeded data	Демо зависит от ручных действий и ломается под стрессом	Сделать seed/reset scripts и не выступать без них
RPC/devnet нестабилен	Подвисают транзакции и UI кажется сломанным	Иметь запасной RPC, pre-funded wallets и clear tx feedback
Settlement evidence неочевиден	Судьи не понимают, как токен связан с активом	Показывать document hash, verifier action и evidence hash как цепочку доказательства

12.2. Предлагаемая структура GitHub репозитория

```
repo/
  programs/receivables_program/
  programs/transfer_hook_program/
  tests/
  apps/web/
  services/api/
  packages/shared-types/
  scripts/seed-demo/
  docs/architecture/
  README.md
  .env.example
```

12.3. Техническая база и официальные ссылки

Ниже - базовые официальные материалы, на которые опирается технический дизайн проекта. Они нужны как reference для реализации, а не как декоративный список ссылок.

- Token Extensions overview - solana.com/docs/tokens/extensions
- Transfer Hook - solana.com/developers/guides/token-extensions/transfer-hook
- Metadata & Metadata Pointer - solana.com/docs/tokens/extensions/metadata
- Program Derived Addresses - solana.com/docs/core/pda
- PDA accounts / invoke_signed - solana.com/docs/core/pda/pda-accounts
- Switchboard Solana / SVM - docs.switchboard.xyz/product-documentation/data-feeds/solana-svm

12.4. Чек-лист перед сдачей

Блок	Что обязательно проверить
On-chain	Program задеплоен в devnet, program IDs зафиксированы, transfer restriction и payout claim работают.
Backend	Upload/hash/metadata flow стабилен; есть seed/reset scripts и читаемые логи.
Frontend	Marketplace, asset detail, portfolio, verifier/admin dashboards отрисовываются на живых данных.
Demo	Есть готовый demo asset, funded wallets, backup RPC и заранее прогнанный сценарий выступления.
Submission	Заполнены README, архитектура, setup steps; подготовлены ссылки для Google Forms и Colosseum.

12.5. Чек-лист за 30 минут до питча

Финальный вывод и предвыступательный контроль

- Ещё раз прогнать полный сценарий: verify -> funding -> restricted transfer -> repayment -> claim.
- Проверить balances всех демо-кошельков и наличие достаточного USDC/lamports.
- Открыть нужные вкладки заранее: marketplace, asset detail, verifier dashboard, admin dashboard, explorer.
- Не менять account model, endpoint names и env-конфиги в день выступления.
- Этот ТЗ уже можно переносить в GitHub Projects / Linear / Notion без дополнительной декомпозиции.